2주차 과제 보고서 — 전처리와 특징 추출

Step 2 · sEMG 팔동작 예측 프로젝트 (손동작 × 팔꿈치 각도 동시 분류)

학 과	컴퓨터소프트웨어학부	학 번	22114553
이 름	전진성	제출일	2026년 10월 15일
저장소	https://github.com/jjs523/semg-gesture-angle (실행 `python main.py week2`, 노트북 `notebooks/week2.ipynb`, 데이터셋 `data/dataset.npz`)		

요약. 1,920시행을 60 Hz 노치 + 20~450 Hz 필터 → mark + 1.0~3.0 s 세그먼트 → 200 ms·100 ms 윈도우 19개 → 시간영역 특징 25차원으로 바꾸고, **시행 단위로 먼저 8:2 분할**한 뒤 학습셋 통계로 표준화하여 학습 29,184 / 테스트 7,296 윈도우의 `dataset.npz`를 만들었다. 학습셋 안의 5-fold 검증에서 세 가지를 확인하였다. ① 윈도우 하나의 정확도는 59.5 %이고 논문 수준(96 %)은 세그먼트(19윈도우) 단위에서 나온다(95.8 %). ② 윈도우를 먼저 만들고 나누면 정확도가 3.9 %p 부풀려진다. ③ **손을 쉬고 있는 4~5 s 구간만으로도 손동작을 87.4 % 맞힌다**(우연 25 %). 클래스마다 별도 파일로 녹음된 데이터라 trial-wise 평가에 녹음 세션 교란이 섞여 있다는 뜻이며, 3주차 해석과 LOSO 검증의 근거가 된다.

1. 단계 1 — 필터 적용과 검증 (제출물 ①)

배포본은 이미 같은 필터를 거친 신호이지만 논문 절차를 재현하기 위해 다시 적용하였다. `iirnotch(60, 30)` 뒤에 4차 Butterworth 20~450 Hz 대역통과를 이어 걸고, 둘 다 `filtfilt`로 시간축(axis=0)에 영위상으로 적용해 신호가 밀리지 않게 하였다. 시행 배열 (1920, 5000, 5) 전체를 한 번에 처리한다 (`src/features.py`의 `preprocess`).

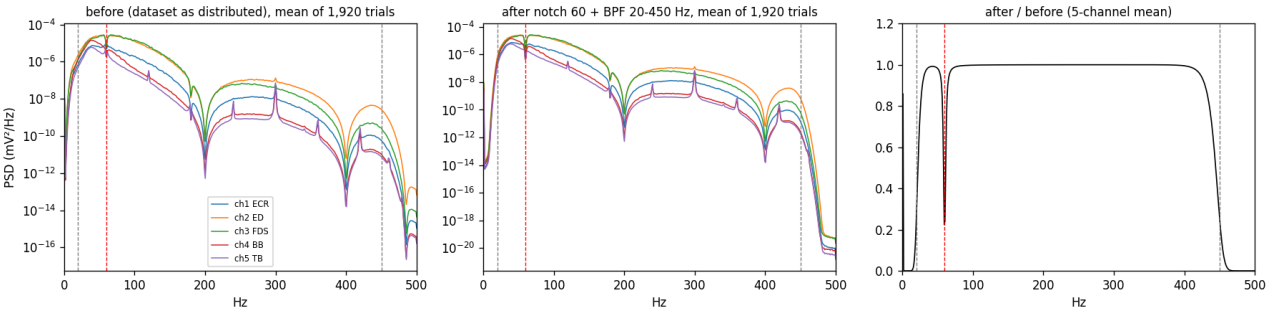


그림 1. 1,920시행 평균 Welch 스펙트럼 — 필터 전(왼쪽), 후(가운데), 후/전 비(오른쪽). 빨간 점선 60 Hz, 회색 점선 20·450 Hz

채널	60 Hz 전력 (후/전)	15 Hz 미만 (후/전)	460 Hz 초과 (후/전)	전체 전력 보존	표준편차 전 (mV)	표준편차 후 (mV)
ch1 ECR	0.203	0.009	0.006	0.910	0.0208	0.0199
ch2 ED	0.236	0.009	0.006	0.945	0.0518	0.0501
ch3 FDS	0.230	0.011	0.006	0.951	0.0377	0.0369
ch4 BB	0.234	0.011	0.007	0.926	0.0119	0.0113
ch5 TB	0.213	0.012	0.009	0.899	0.0049	0.0044

표 1. 필터 효과 (1,920시행 평균 스펙트럼 기준, 표준편차는 피험자 a·30_fist 첫 시행)

60 Hz 성분은 20~24 %로 줄고, 15 Hz 미만과 460 Hz 초과 성분은 1 % 남짓만 남는다. 전체 전력은 90~95 %가 보존되어 근활성 대역은 손대지 않았다. 다만 그림 1 왼쪽에서 보듯 배포본에는 이미 60 Hz와 200·400 Hz 골이 있고, **120·180·300 Hz 같은 전원 고조파 봉우리는 노치 하나로 지워지지 않고 남는다**. 논문 조건(60 Hz 노치 하나)을 그대로 따랐으므로 이 고조파는 특징에 그대로 들어가며, 고조파 노치 추가는 3주차 이후 어블레이션 후보로 남긴다.

2. 단계 2·3 — 분석 구간과 슬라이딩 윈도우 (제출물 ②)

논문은 2 s 세그먼트의 시작점을 명시하지 않고 실습 자료는 1.5 s를 가정하지만, 1주차에 1,920시행의 RMS 포락선으로 측정한 활성 구간은 mark 뒤 0.72 s 시작 · 1.68 s 최대 · 3.22 s 종료(중앙값)였다. 1.5~3.5 s로 자르면 상승부 절반을 잃으므로 **mark + 1.0~3.0 s**를 세그먼트로 잡고 재현 가정으로 기록한다. 이 2,000샘플을 200샘플 윈도우, 100샘플 증분으로 자르면 (2000 − 200) / 100 + 1 = 19개가 나온다.

```
[단계 2] 세그먼트 모양 (2000, 5)  (mark + 1.0~3.0 s)
[단계 3] 윈도우 배열 (19, 200, 5)  ((2000-200)/100+1 = 19)
```

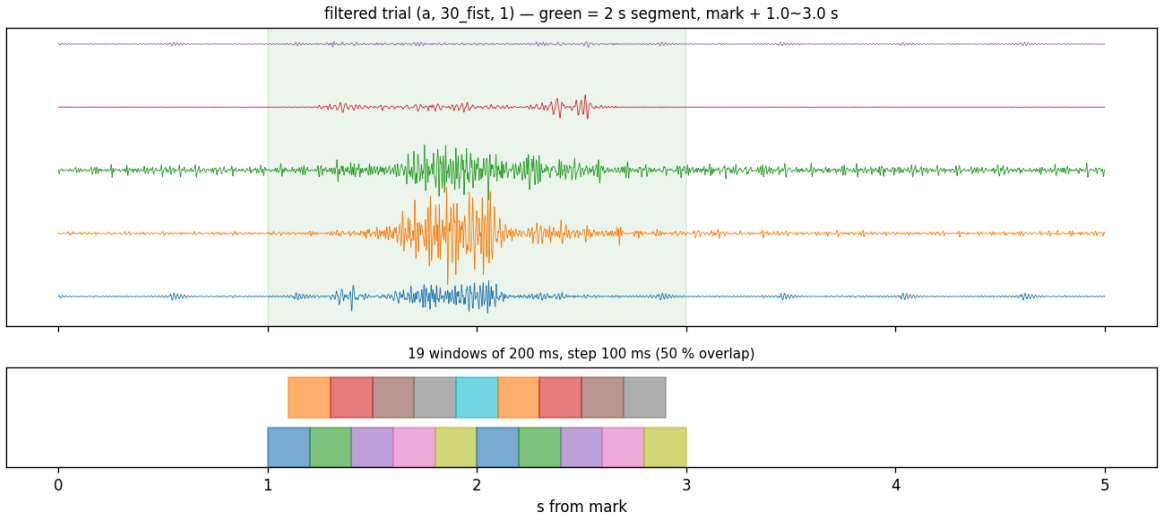


그림 2. 필터된 시행 하나(5채널)와 2 s 세그먼트(녹색), 그 안의 200 ms 윈도우 19개

3. 단계 4 — 시간영역 특징 구현 (제출물 ③)

윈도우 하나에서 채널마다 RMS·MAV·WL·ZC·SSC를 numpy만으로 계산해 25차원 벡터를 만든다. 논문 식 (4)에 따라 **ZC에만 임계값 0.01**을 두고, 식 (5)의 SSC는 $(x_i - x_{i-1})(x_i - x_{i+1}) > 0$인 점의 개수로 임계값 없이 센다. 순서는 [RMS×5채널, MAV×5, WL×5, ZC×5, SSC×5]로 고정하였다. 함수는 (…, 200, 5) 모양을 받아 시행 배열 전체에 한 번에 적용된다.

```
def td_features(w, th=ZC_TH):
    """w: (..., win, C) -> (..., 5*C). 마지막 축 순서 = [RMS x C, MAV x C, WL x C, ZC x C, SSC x C]"""
    d = np.diff(w, axis=-2)
    rms = np.sqrt(np.mean(w ** 2, axis=-2))
    mav = np.mean(np.abs(w), axis=-2)
    wl = np.sum(np.abs(d), axis=-2)
    zc = np.sum((w[..., :-1, :] * w[..., 1:, :] < 0) & (np.abs(d) >= th), axis=-2)           # 식 (4): 임계값 th
    ssc = np.sum((w[..., 1:-1, :] - w[..., :-2, :]) * (w[..., 1:-1, :] - w[..., 2:, :]) > 0, axis=-2)  # 식 (5): 임계값 없음
    return np.concatenate([rms, mav, wl, zc, ssc], axis=-1).astype(np.float32)
```

```
[단계 4] 특징 벡터 (25,)
```

채널	ZC 평균 (th 0.01)	ZC 평균 (th 0)	SSC 평균
ECR	6.65	26.90	40.37
ED	12.62	28.92	42.00
FDS	12.25	30.81	40.57
BB	4.20	20.82	33.56
TB	2.78	24.72	37.19

표 2. 채널별 ZC·SSC 평균 (윈도우 200샘플당 횟수, 전체 36,480윈도우)

임계값 0.01은 이 데이터의 단위(mV)에서 상당히 크다. 휴식 수준 진폭이 0.005 mV 안팎이라 임계값 0에서는 윈도우당 21~31회인 영교차가 0.01에서는 3~13회로 줄고, 전체 윈도우·채널의 29.5 %에서 ZC가 0이다. 즉 이 설정의 ZC는 "주파수"보다 "충분히 큰 진동이 있었는가"를 세는 특징에 가깝다. 논문 원자료의 단위가 확인되지 않아 식대로 0.01을 유지하였다.

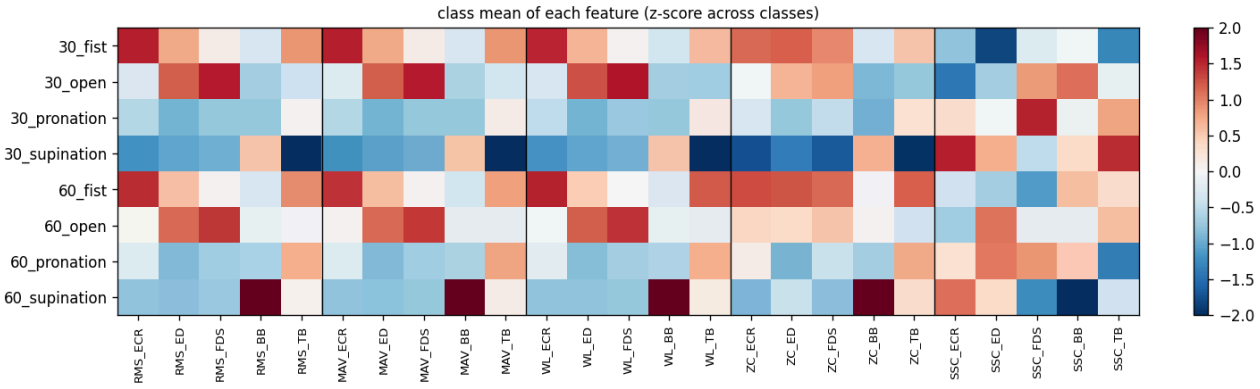


그림 3. 25개 특징의 클래스별 평균 (클래스 간 z-score)

그림 3에서 동작 축은 전완 채널 블록이 가른다(주먹은 ECR, 펴기는 ED·FDS가 높음). 각도 축은 BB 열에서 갈려 60° 회외가 RMS·MAV·WL·ZC·SSC 모두 극단값이다. 1주차의 진폭 비교(회외 BB +55 %)가 특징 공간에서도 그대로 보인다.

4. 단계 5 — 데이터셋 만들기, 누수 막기 (제출물 ④·⑤)

한 시행의 19개 윈도우는 절반씩 겹치므로, 윈도우를 먼저 만들고 나누면 이웃 윈도우가 학습·테스트 양쪽에 들어간다. 그래서 **시행(mark 1개) 단위로 먼저 8:2로 나누고, 각 집합의 시행 안에서만 윈도우와 특징을 만들었다**(노란 줄). 층화는 클래스 × 피험자(64칸, 칸마다 30시행 → 24 / 6)로 걸어 8클래스와 8명이 양쪽에 똑같이 들어가게 했고, 시드 42로 고정하였다. 분할된 시행 목록은 `results/week2/split.json`에 있다.

```
strat = (meta.label + "_" + meta.participant).values        # 클래스 x 피험자 층화
# ① 시행(=mark 1개) 단위로 먼저 8:2 분할  ← 순서가 핵심
tr_idx, te_idx = train_test_split(np.arange(len(meta)), test_size=0.2,
                                  stratify=strat, random_state=42)
assert not set(tr_idx) & set(te_idx)                        # 같은 시행이 양쪽에 없음
# ② 각 집합의 시행 안에서만 세그먼트 -> 윈도우 -> 특징 생성
def build(idx):
    Xw = F.trial_features(Xf[idx])                          # (n시행, 19, 25)
    y  = np.repeat(meta.label.values[idx], Xw.shape[1])
    tid = np.repeat(idx, Xw.shape[1])                       # 윈도우가 속한 시행 번호
    return Xw.reshape(-1, 25), y, tid
Xtr, ytr, tid_tr = build(tr_idx)
Xte, yte, tid_te = build(te_idx)
assert not set(tid_tr) & set(tid_te)
# ③ 표준화 통계는 학습셋에서만
sc = StandardScaler().fit(Xtr)
Xtr_s, Xte_s = sc.transform(Xtr), sc.transform(Xte)
```

	학습 시행	테스트 시행	학습 윈도우	테스트 윈도우
30_fist	192	48	3648	912
30_open	192	48	3648	912
30_pronation	192	48	3648	912
30_supination	192	48	3648	912
60_fist	192	48	3648	912
60_open	192	48	3648	912
60_pronation	192	48	3648	912
60_supination	192	48	3648	912
합계	1536	384	29184	7296

표 3. 클래스별 학습·테스트 시행 수와 윈도우 수 (제출물 ⑤)

	a	b	c	d	e	f	g	h
학습	192	192	192	192	192	192	192	192
테스트	48	48	48	48	48	48	48	48

표 4. 피험자별 시행 수

학습 1,536시행 × 19 = 29,184윈도우, 테스트 384시행 × 19 = 7,296윈도우로 정확히 4 : 1이고 8클래스·8명이 모두 균등하다. 논문의 1,536 / 384 세그먼트 분할과 같은 규모다.

5. 단계 6 — 표준화와 저장

`StandardScaler`를 학습셋에만 `fit`하고 테스트셋에는 `transform`만 적용했다. 학습셋은 25개 특징 모두 평균 |최대| 8.3e-06, 표준편차 0.9999~1.0000로 0 · 1이 되었고, 테스트셋은 평균 |최대| 0.028, 표준편차 0.966~1.039로 0 · 1에 가깝지만 정확히 같지 않다(정상). `data/dataset.npz`(2.5 MB, 저장소 포함)에는 `Xtr, ytr, Xte, yte` 외에 윈도우가 속한 시행 번호·윈도우 위치·피험자, 특징 이름, 스케일러 통계를 함께 저장해 3주차에 세그먼트 단위 집계와 피험자별 분석을 할 수 있게 했다. 5-fold용으로는 표준화 전 특징 (1920, 19, 25)을 `data/features_all.npz`에 따로 저장했다(폴드마다 학습 폴드에 다시 fit).

6. 설정 근거 — 윈도우 200 ms · 오버랩 50 % (제출물 ⑥)

근거를 말로만 쓰지 않으려고, **학습 1,536시행 안에서만** 시행 단위 5-fold로 RF(트리 100개)를 돌려 설정을 비교했다. 테스트 384시행은 3주차 최종 평가까지 보지 않는다.

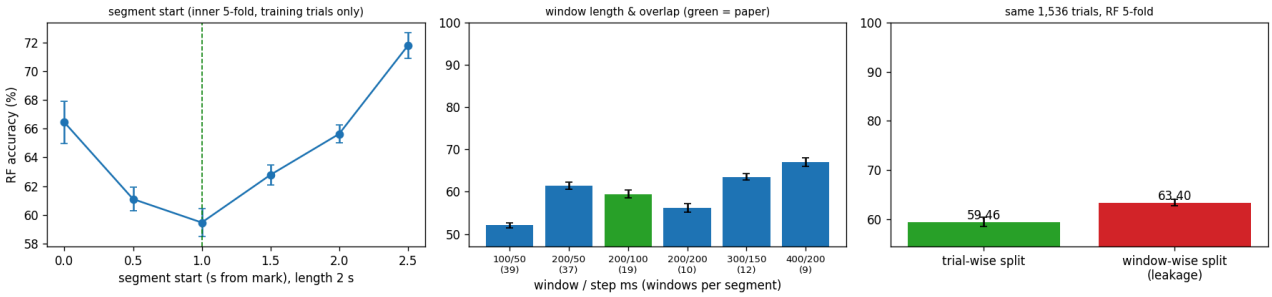


그림 4. 세그먼트 시작점(왼쪽), 윈도우 길이·증분(가운데), 분할 방식(오른쪽)에 따른 윈도우 정확도. 학습셋 안 5-fold

설정	오버랩	세그먼트당 윈도우	정확도
100 ms / 50 ms	50 %	39	52.05 ± 0.65
200 ms / 50 ms	75 %	37	61.48 ± 0.82
200 ms / 100 ms	50 %	19	59.46 ± 0.96
200 ms / 200 ms	0 %	10	56.22 ± 1.01
300 ms / 150 ms	50 %	12	63.53 ± 0.70
400 ms / 200 ms	50 %	9	66.95 ± 1.00

표 5. 윈도우 길이·증분 비교 (윈도우 단위 정확도 %, 세그먼트 1.0~3.0 s)

윈도우 200 ms. 근전도는 수백 ms 안에서 통계적 성질이 거의 변하지 않는다고 볼 수 있어 RMS·ZC 같은 통계량이 안정적으로 잡히고, 의수·재활 기기 제어에서 허용되는 판정 지연(약 300 ms) 안에 들어오는 길

이다. 실제로 100 ms로 줄이면 정확도가 52.1 %로 크게 떨어지고 300·400 ms는 63.5·67.0 %로 오르지만, 지연이 커지고 세그먼트당 표본이 12·9개로 줄어드므로 논문 설정 200 ms를 유지했다.

오버랩 50 %(증분 100 ms). 증분 100 ms는 판정을 0.1 s마다 갱신하게 하고, 같은 2 s에서 표본을 10개(오버랩 0 %)에서 19개로 두 배 가까이 늘린다. 오버랩 0 %는 56.2 %, 75 %(증분 50 ms)는 61.5 %로 오버랩이 늘수록 조금씩 오르지만, 75 %는 이웃 윈도우가 거의 같은 정보라 이득 2.0 %p에 비해 계산량과 표본 간 상관이 두 배가 되므로 50 %로 두었다.

7. 추가 검증 — 숫자를 믿기 전에 확인한 것

7.1 표본 단위: 논문 96 %는 세그먼트 단위다

표본 단위	정확도
윈도우 1개 (200 ms)	59.46 ± 0.96
세그먼트: 19윈도우 확률 평균	95.77 ± 1.11
세그먼트: 19×25 = 475차원 연결	95.70 ± 1.21
세그먼트: 윈도우 평균 25차원	89.58 ± 1.48

표 6. 같은 특징, 다른 표본 단위의 정확도 (RF, 학습셋 안 5-fold, %)

윈도우 하나(200 ms)로 8클래스를 맞히면 59.5 %에 그치지만, 세그먼트의 19윈도우 확률을 평균하면 95.8 %, 19 × 25 = 475차원으로 이어 붙이면 95.7 %가 되어 논문의 RF 96.72 %와 같은 수준이다. 논문의 테스트 384개도 세그먼트 수와 정확히 같으므로, **논문 수치는 세그먼트 단위 평가로 보는 것이 맞다**. 3주차에는 윈도우 단위와 세그먼트 단위를 둘 다 보고하고, 논문과는 세그먼트 단위로 비교한다.

7.2 누수의 크기: 윈도우부터 나누면 3.9 %p 부풀려진다

분할 방식	정확도
시행 단위 (올바름)	59.46 ± 0.96
윈도우 단위 (누수)	63.40 ± 0.60

표 7. 같은 학습 1,536시행을 두 방식으로 5-fold (윈도우 정확도 %)

윈도우를 먼저 만들고 섞어서 나누면, 50 % 겹치는 이웃 윈도우가 학습·평가 양쪽에 들어가 정확도가 59.46 → 63.40 %로 오른다. 4절의 순서(시행 먼저)가 이 차이를 막는다.

7.3 녹음 세션 교란: 쉬고 있는 손으로도 동작을 맞힌다

그림 4 왼쪽에서 이상한 점이 보였다. 세그먼트를 동작 구간(1.0~3.0 s)에서 휴식 구간(2.5~4.5 s) 쪽으로 옮길수록 정확도가 **올라간다**(59.5 → 71.8 %). 동작 정보가 줄어드는데 정확도가 오른다면 동작이 아닌 다른 단서가 있다는 뜻이라 직접 확인했다.

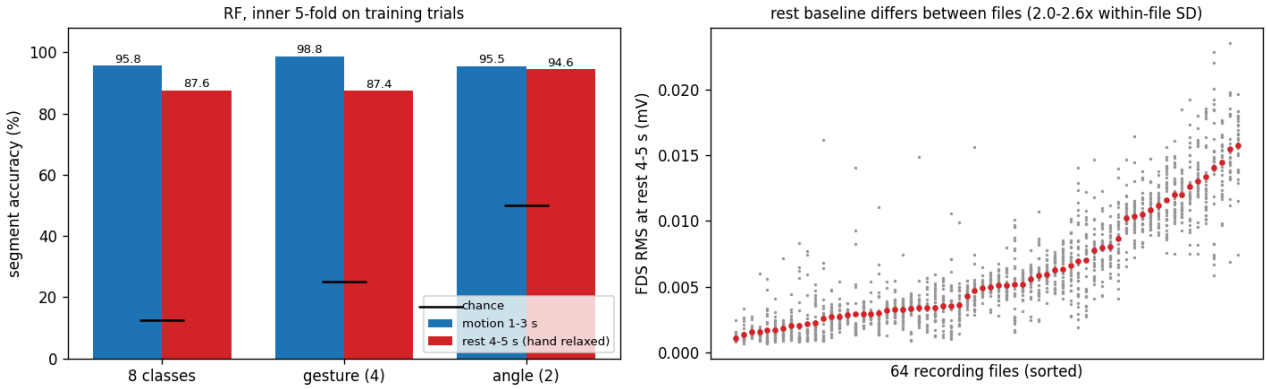


그림 5. 왼쪽: 동작 구간과 휴식 구간(4~5 s) 특징으로 각 목표를 맞힌 세그먼트 정확도(검은 선 = 우연 수준). 오른쪽: 64개 녹음 파일의 휴식 구간 FDS RMS(회색 = 시행, 빨강 = 파일 평균)

	우연 수준	동작 1.0~3.0 s	휴식 4.0~5.0 s
8클래스	12.5	95.77	87.56
손동작 4종	25.0	98.76	87.43
각도 2종	50.0	95.51	94.60

표 8. 동작 구간 vs 휴식 구간으로 맞힌 정확도 (세그먼트 단위, 학습셋 안 5-fold, %)

채널	파일 간 SD (mV)	파일 내 SD (mV)	비율
ECR	0.00212	0.00094	2.27
ED	0.00401	0.00156	2.57
FDS	0.00410	0.00157	2.61
BB	0.00234	0.00114	2.04
TB	0.00231	0.00090	2.55

표 9. 휴식 구간(4~5 s) RMS의 파일 간 / 파일 내 표준편차

손을 쉬고 있는 4~5 s 구간만으로 손동작 4종을 87.4 % 맞힌다(우연 25 %). 각도(94.6 %)는 휴식 중에도 팔꿈치 자세를 유지하므로 정당한 단서가 일부 있지만, 손동작은 휴식 구간에 정보가 없어야 한다. 원인은 데이터 구조에 있다. 이 데이터는 **클래스 × 피험자마다 별도 파일로 30시행을 연속 녹음**했고, 표 9처럼 휴식 기저선 크기가 파일마다 2.0~2.6배 더 크게 달라진다(전극 접촉·피부 상태가 녹음마다 다름). 시행 단위로 무작위 분할하면 같은 파일의 시행이 학습과 평가에 함께 들어가므로, 모델은 동작 패턴과 함께 "이 녹음 세션의 기저선"을 단서로 쓸 수 있다.

이 결과가 3주차에 주는 의미는 세 가지다. ① 논문과 같은 trial-wise 정확도(96 % 수준)에는 녹음 세션 단서가 섞여 있으므로 "동작을 그만큼 잘 읽는다"로 해석하면 안 된다. ② 피험자를 통째로 빼는 LOSO 검증은 평가 피험자의 녹음 파일을 전혀 보지 않으므로 이 교란을 제거한 성능을 준다. ③ 세그먼트 시작점을 정확도로 고르면 교란을 최대화하는 쪽(휴식 구간)을 고르게 되므로, 1.0~3.0 s는 정확도가 아니라 1주차의 생리적 활성 구간으로 정한 것이 맞다.

8. 체크포인트·제출물 대조

슬라이드 17쪽 제출물	산출물
① 필터 검증	그림 1, 표 1 — results/week2/filter_check.png
② 윈도우 확인	2절 출력 (19, 200, 5) — 노트북 단계 3
③ 특징 구현 코드	3절 td_features 전체 — src/features.py
④ 분할 코드 (시행 먼저)	4절 노란 표시 — src/week2_features.py 단계 5, results/week2/split.json
⑤ 데이터셋 요약	표 3·4 — results/week2/dataset_summary.csv
⑥ 설정 근거 (200 ms · 50 %)	6절 두 문단, 표 5 — results/week2/window_length.csv
제출 형식	이 PDF + data/dataset.npz + notebooks/week2.ipynb + 저장소 링크

표 10. 제출물 대응표

강의 자료 15쪽 체크포인트 7개 — 필터 후 5채널 표준편차 출력, 세그먼트 (2000, 5), 윈도우 (19, 200, 5), 특징 (25,), 시행 단위 분할 후 특징 생성, 학습셋 평균 0 · 분산 1, `dataset.npz` 저장 — 는 모두 노트북 실행본(`notebooks/week2_executed.ipynb`)에서 확인된다.